



1	 Word Frequency	Take a sentence and create a dictionary containing the frequency of each word.	Enter a sentence: this is a test this is simple test	Output: {'this': 2, 'is': 2, 'a': 1, 'test': 2, 'simple': 1}
---	---	--	---	---

In [18]: word = 'Good life and a Good day is a best life'

```
def wordfreq(word):  
    word = word.replace(',', '  
    l = word.split(' '  
    s = set(l)  
    print(l)  
    print(s)  
  
    d = {}  
  
    for word in s:  
        d[word] = 0  
  
    print('\n',d)  
  
    for key in s:  
        for word in l:  
            if(key == word):  
                d[key]+=1  
  
    print('\n', d)  
  
    return d  
  
x = wordfreq(word)
```

```
['Good', 'life', 'and', 'a', 'Good', 'day', 'is', 'a', 'best', 'life']  
{'is', 'day', 'Good', 'life', 'and', 'a', 'best'}
```

```
{'is': 0, 'day': 0, 'Good': 0, 'life': 0, 'and': 0, 'a': 0, 'best': 0}
```

```
{'is': 1, 'day': 1, 'Good': 2, 'life': 2, 'and': 1, 'a': 2, 'best': 1}
```

In [23]: #2

```
list1 = [1,2,3,4,5]  
list2 = [3,4,5,6,7]  
  
def common(list1, list2):  
    res = []  
  
    for ele in list1:  
        if(ele in list2):  
            res.append(ele)  
  
    return res  
  
common(list1, list2)
```

Out[23]: [3, 4, 5]

```
In [22]: # 3 -> Student Management
# Exception Handling

sd = {}

def adds():
    global sd

    print('-----')
    name = input('Enter Student\'s Name : ')
    marks = int(input('Enter Student\'s Marks : '))

    sd[name] = marks

    print(sd)
    print('--Student Added Successfully--')

def search():
    global sd

    print('-----')
    name = input('Enter Student\'s Name to Search: ')

    try:
        print(f'{name} : {sd[name]}')

    except KeyError:
        print('--Student Record Doesn\'t Exist--')

def update():
    global sd

    print('-----')
    name = input('Enter Student\'s Name to Update: ')
    marks = int(input('Enter New Marks : '))

    try:
        sd[name] = marks
        print('--Student Record is Updated--')

    except KeyError:
        print('--Student Record Doesn\'t Exist--')

def delete():
    global sd

    print('-----')
    name = input('Enter Student\'s Name to Delete: ')

    try:
        del sd[name]
        print('--Student Record is Deleted--')
```

```

except KeyError:
    print('--Student Record Doesn\'t Exist--')

def topper():
    global sd

    print('-----')

    if(sd == {}):
        print('--Empty--')

    else:
        mlist = list(sd.values())
        mlist.sort()
        topmarks = mlist[-1]

        for key in sd:
            if(sd[key] == topmarks):
                print(f'Topper -> {key} : {sd[key]}')

topper()

```

```

-----
--Empty--

```

In [23]: # Student Management

```

while(True):
    print('-----')

    print('-- STUDENT MANAGEMENT MENU --')
    print('1. Add a New Student')
    print('2. Update Student Marks')
    print('3. Delete Student Record')
    print('4. Find Highest Scorer')
    print('5. Search Student Record')
    print('0. STOP')

    o = int(input('Enter your choice : '))

    if(o == 0):
        break

    elif(o == 1):
        adds()

    elif(o == 2):
        update()

    elif(o == 3):

```

```

        delete()

    elif(o == 4):
        topper()

    elif(o == 5):
        search()

    else:
        print('--INVALID--')

```

```

-----
-- STUDENT MANAGEMENT MENU --

```

1. Add a New Student
2. Update Student Marks
3. Delete Student Record
4. Find Highest Scorer
5. Search Student Record
0. STOP

```

-----
--Student Record Doesn't Exist--
-----

```

```

-- STUDENT MANAGEMENT MENU --

```

1. Add a New Student
2. Update Student Marks
3. Delete Student Record
4. Find Highest Scorer
5. Search Student Record
0. STOP

```

-----
--Empty--
-----

```

```

-- STUDENT MANAGEMENT MENU --

```

1. Add a New Student
2. Update Student Marks
3. Delete Student Record
4. Find Highest Scorer
5. Search Student Record
0. STOP

```

-----
{'Shreyas R S': 86}

```

```

--Student Added Successfully--
-----

```

```

-- STUDENT MANAGEMENT MENU --

```

1. Add a New Student
 2. Update Student Marks
 3. Delete Student Record
 4. Find Highest Scorer
 5. Search Student Record
 0. STOP
- ```

```

Shreyas R S : 86

```

-- STUDENT MANAGEMENT MENU --
1. Add a New Student
2. Update Student Marks
3. Delete Student Record
4. Find Highest Scorer
5. Search Student Record
0. STOP
```

```

--Student Record is Updated--

```

```

-- STUDENT MANAGEMENT MENU --
1. Add a New Student
2. Update Student Marks
3. Delete Student Record
4. Find Highest Scorer
5. Search Student Record
0. STOP
```

Shreyas R S : 90

```

-- STUDENT MANAGEMENT MENU --
1. Add a New Student
2. Update Student Marks
3. Delete Student Record
4. Find Highest Scorer
5. Search Student Record
0. STOP
```

```

{'Shreyas R S': 90, 'Dhruthi': 95}
--Student Added Successfully--

```

```

-- STUDENT MANAGEMENT MENU --
1. Add a New Student
2. Update Student Marks
3. Delete Student Record
4. Find Highest Scorer
5. Search Student Record
0. STOP
```

```

Topper -> Dhruthi : 95

```

```

-- STUDENT MANAGEMENT MENU --
1. Add a New Student
2. Update Student Marks
3. Delete Student Record
4. Find Highest Scorer
5. Search Student Record
0. STOP

```

--Student Record is Deleted--

-----  
-- STUDENT MANAGEMENT MENU --

1. Add a New Student
2. Update Student Marks
3. Delete Student Record
4. Find Highest Scorer
5. Search Student Record
0. STOP

-----  
--Student Record Doesn't Exist--

-----  
-- STUDENT MANAGEMENT MENU --

1. Add a New Student
2. Update Student Marks
3. Delete Student Record
4. Find Highest Scorer
5. Search Student Record
0. STOP

In [ ]: